Idiomatic C to Rust Conversion Using Fine-tuned Large Language Models

Eder Martinez, Henry Glenn, and Miguel Arellano
Computer Science
Kansas State University
2900 College Ave, Manhattan, KS

Abstract

When considering how to create a tool capable of converting code from one programming language to another, a number of factors come into play when determining the best approach. These factors mostly pertain to how similar the languages are and in what specific areas they differ from each other. The most crucial factors are how differently the two languages handle memory management, data types, code structure, and concurrency. For languages that utilize identical or very similar memory management and code structure paradigms, near-perfect conversion can be achieved through deterministic algorithms, such as abstract syntax tree transformation and lexical analysis. Even if two languages differ dramatically in their syntax, all that matters for idiomatic deterministic conversion is that those languages “think” similarly. However, when the underlying memory management and structure paradigms differ greatly between two languages, deterministic attempts at conversion result in the mechanics of the source languages being ported to the destination language with no regard for the fundamental paradigm shift between the two. As a result, the converted code is inefficient and sometimes even hazardous, making a deeper and more understanding form of translation necessary. This is why, for code conversion between two deeply distinct programming languages, fine-tuned LLMs are utilized in order to comprehend and translate not just surface-level syntax differences but also deeper idiomatic and paradigm differences.

1. Introduction

For this project, our team's chosen purpose was to solve the problem of file-level C to Rust code conversion using supervised fine-tuning of a transpiler LLM. Unlike deterministic conversion algorithms that already exist, our goal was to have our AI system be able to convert C code to safe idiomatic Rust code, rather than just trying to produce a translation that achieves identical functionality. What this means practically is that the translated Rust code our fine-tuned AI system produces should never contain `unsafe` functions and code blocks, Foreign Function Interface (FFI) types, or manual memory management. The reason our team chose to solve this particular problem was that very few C to safe Rust code transpilers exist, with many of the

deterministic conversion algorithms simply using wall-to-wall unsafe Rust code in their translations. Hence, by attempting to solve this problem, we have at the very least brought future research closer to an open-source solution. The official methodological pillar for our project is Natural Language, and our official track is Machine Learning and Probabilistic Models.

2. Background & Related Work

Through the course of this project, our team used a number of discoveries made by previous research teams to aid us in our mission to achieve file-level C to safe idiomatic Rust code conversion. Here, we will mention the previous AI research we utilized and how it assisted our project.

2.1 LoRa

Using modern methods, we aimed to fine-tune our model in order to come up with the most accurate results possible. We performed fine-tuning using Low-Rank Adaptation, LoRA for short. LoRA is a technique first introduced by Microsoft **[1]**, which allows us to perform fine-tuning of a pretrained model in a way that is less computationally intensive compared to more traditional methods. Compared to generalized fine-tuning, which "allows the training of a subset of pre-trained parameters" **[1]**, LoRA allows for increased efficiency. As the number of parameters grows, LoRA roughly follows the training size of the original model while updating the weights. This allows us to apply LoRA to any subset of weights in a trainable neural network and, in turn, reduce the number of parameters that we must concern ourselves with when fine-tuning a model. Using this method, we see a decrease in memory usage. For example, when switching from a more traditional fine-tuning approach to LoRA, there were benefits of reducing the size of computational cost by roughly 10,000 times **[1]**. All these benefits equate to a 4 times reduction in the use of VRAM. This reduction in VRAM usage allows us to train using consumer hardware. Due to this reduction, we use fewer GPUs, which can help reduce financial cost. Another benefit is the fact that we can save time when training, too.

2.2 CodeT5+

CodeT5+ is an encoder-decoder model. Encoder models focus on code search and bug detection, while decoders excel at writing code and autocomplete. The CodeT5+ model attempts to do both. CodeT5+ is capable of multiple modes: encoder only, decoder only, or both encoder and decoder modes **[2]**. Due to the features CodeT5 offers, we theorized we could use the two parts of it to do the following. The encoder would attempt to understand the incoming code, find bugs and patterns, while the decoder would focus on the code generation with the given input **[2]**. This gave CodeT5 a theoretical advantage compared to other models that solely focused on a single way of operation.

2.3 Qwen2.5 Coder

Another architecture that we identified to have notable potential was Qwen2.5 Coder. This code-specific model was trained on a vast amount of tokens. Using over 5.5 trillion tokens, composed of both synthetic and general data, it demonstrates impressive code generation capabilities. Capabilities that at times surpass state-of-the-art models **[3]**. Qwen2.5 Coder was

trained using a large, heavily curated dataset and using tokenization techniques to better understand code, end-of-code, and prefixes **[3]**. Qwen also uses filters, which are built using smaller models; these filters focus on reducing unnecessary semantic complexity, compared to using larger models. Finally, to avoid hallucination when training on synthetic data, CodeQwen2.5 Coder uses executor validation, ensuring only executable code is returned when creating synthetic data. Due to all these advantages, we theorized that Qwen2.5-Coder had huge potential to be fine-tuned and be a great choice for our desired C-to-Rust translator.

3. Methodology

3.1 Data Collection

We began our project by first searching for large human-reviewed datasets containing C code and equivalent Rust code pairs. Our goal was to amass a dataset of at least 5,000 entries, each containing high-quality C code and equivalent Rust code. While many conversion datasets existed for other, more popular programming languages, the only quality C to Rust translation datasets we identified were CodeTransOcean's niche translation dataset and RosettaCode's C and Rust algorithms dataset. While we initially thought the CodeTransOcean niche translation dataset would contain a sufficient number of C and Rust code pairs, after sanitizing the data, we discovered that more than 90% of the entries in the dataset were either duplicates or contained code that did not compile due to syntax errors. In the end, we were only able to extract about 900 high-quality C and Rust code pairs. In response to this, we also utilized the RosettaCode dataset to bring the total number of entries in our dataset to 1,330. While we knew this small of a final dataset would likely not be sufficient to fully convey the C to Rust conversion skill to an LLM, we spent a month trying to find suitable alternatives, but to no avail. Hence, we then moved on to sanitizing our final dataset.

3.2 Data Sanitization

Our data sanitization process cleaned new entries in three main steps. First, it took an entry from one of the original datasets, extracted both the C and Rust code snippets, and then removed any code comments present. Second, it checked if the C code was already present in the final dataset; if so, the entry was dropped. Third, we ran a compiler check on both the C and Rust code to ensure they both built. If the Rust code caused a "cargo package not installed" compiler error, then the necessary cargo packages were installed, and the compiler check was rerun. If either of the code snippets failed to build, the entry containing them was dropped. Finally, after sanitation, we added an attribute to every entry in the final dataset containing a best-effort translation of the C code in that entry into unsafe Rust code using the open-source C2Rust transpiler.

3.3 Model Selection

Next, we moved on to selecting a language model to fine-tune. Before we started our model search, we first identified characteristics our base model should have. We came up with three critical characteristics that our base model should have in order to best achieve our goal. First, our base model should be as small as possible to both keep memory usage low and also allow

our small dataset to be effective. Second, our base model should use an architecture effective for code-to-code tasks. Third, our base model should already be able to understand and write code in both the C and Rust programming languages. Once these search parameters were defined, we set out to find models that fit these criteria, where two models emerged as potential candidates.

The first model we found was CodeT5+ by Salesforce. We thought this model series was ideal for our purposes because it had a 2 billion parameter variant, it used an encoder-decoder architecture ideal for conversion tasks, and it already understood 9 programming languages **[2]**. However, when we started to attempt fine-tuning the CodeT5 models, we ran into a variety of issues. First, many of the CodeT5 models had misconfigured tokenizers that caused the Python libraries we were using to fine-tune our models to throw errors we were unequipped to fix. Second, the CodeT5+ model series only understands C and not Rust, something we initially overlooked. This caused the models we were able to fine-tune to hallucinate C syntax into the Rust conversions. Third, the CodeT5+ model series wasn't originally intended to be finetuned using LoRA **[2]**, and instead was intended to be finetuned with a method called shallow tuning. As a result, LoRA finetuning was extremely difficult to set up and required several workarounds to function with modern Python libraries.

The second model we found was Qwen2.5 Coder by Alibaba. We ended up using this model because it has smaller model variants, uses a standard decoder-only architecture, which is acceptable for translation tasks, and it can already understand and write code in 92 programming languages, including both C and Rust **[3]**.

3.4 Baseline Architecture

For our baseline model, we are using the 7.61 billion parameter instruct variant of Qwen2.5 Coder (Qwen2.5-Coder-7B-Instruct). This model is a decoder-only transformer that utilizes grouped query attention (GQA) and rotary positional encodings (RoPE). When prompting this model for baseline conversions, all that is provided is a simple instruction before the C code snippet: "Convert this C code to Rust."

3.5 Proposed Architecture

For our proposed model, we used the 7.61 billion parameter instruct variant of Qwen2.5 Coder (Qwen2.5-Coder-7B-Instruct) as a foundation, then augmented it with a LoRA module. The LoRA adapter we devised targets the linear projections of the model's attention mechanism (query, key, value, and output matrices), as well as the feed-forward network projections (gate, up, and down matrices). The rank of the LoRA adapter was set to 16, the scaling factor was set to 32, and the dropout rate was set to 0.05 to prevent overfitting. Additionally, the proposed model is given a list of constraints in a system instruction to follow while translating. This list of constraints is present during both fine-tuning and inference and is as follows:

1. Prefer 100% safe code. Use `unsafe` only if it is fundamentally impossible to achieve the behavior otherwise.

2. Use standard Rust types (`i32`, `usize`, `str`, `String`, etc.). Strictly avoid FFI types (`c_int`, `c_char`, `c_void`, `*mut T`).
3. Translate C-style manual memory management, pointer arithmetic, and raw arrays into Rust's ownership model (e.g., `Vec<T>`, `Box<T>`, slices, and iterators).
4. Refactor C-style error codes (e.g., returning `-1` or `NULL`) into idiomatic `Result<T, E>` or `Option<T>` patterns.
5. Do not use any external crates unless they are commonly used. Rely almost entirely on `std`.
6. Do not include any code comments in your output.
7. Replace `__asm__` with safe equivalents or `core::arch::asm!`.
8. Preserve original names but adjust to snake_case/PascalCase standards.
9. Respond strictly with a single markdown code block tagged `rust`, avoiding any conversational filler, explanations, or extraneous text.

During the fine-tuning stage, examples were padded so that all examples in our dataset could utilize tensors of the same dimensions. Furthermore, a completion-only masking strategy was employed during the training process to ensure the optimizer only calculated loss for generated Rust code and not for the system instruction, the input C code, or padding tokens.

4. Experiments & Rigorous Evaluation

4.1 Evaluation Description

To test our baseline and proposed models, we employed a cross-validation setup with five seed variations and two folds (5x2cv paired t-test). For our dataset of 1,330 examples, this means we split our data into two folds of 665 examples and used the five randomly selected seeds 42, 1337, 2024, 777, and 999. For our test metrics, we used ChRF for simple character-level similarity, CodeBLEU for syntax and code structure similarity, CodeBERT F1 for code semantic similarity, and Pass@1 compilation rate as a binary measure of code correctness.

For the statistical rigor evaluation, we defined two models, M_B and M_P. Let M_B be the baseline model with μ_B representing the true mean performance of the baseline across the dataset. Let M_P be the proposed model with μ_P representing the true mean performance of the proposed alternative across the dataset. Since the goal of this project is to prove that our proposed model is an improvement over the baseline, we will use a one-sided test to maximize the statistical power of the result. For this test, we define a null hypothesis H_0 and an alternative hypothesis H_A. Let H_0 represent the outcome where the proposed model is no better than the baseline and $\mu_P \leq \mu_B$. Let H_A represent the outcome where the proposed model is strictly better than the baseline and $\mu_P > \mu_B$.

4.2 Evaluation Results

To prove or disprove H_0, we compare the *p*-values generated by the Dietterich 5x2cv statistical test to a rigorous significance threshold of **α**. For this evaluation, we set **α** = 0.01. If a *p*-value is less than **α**, then it is considered a significant improvement.

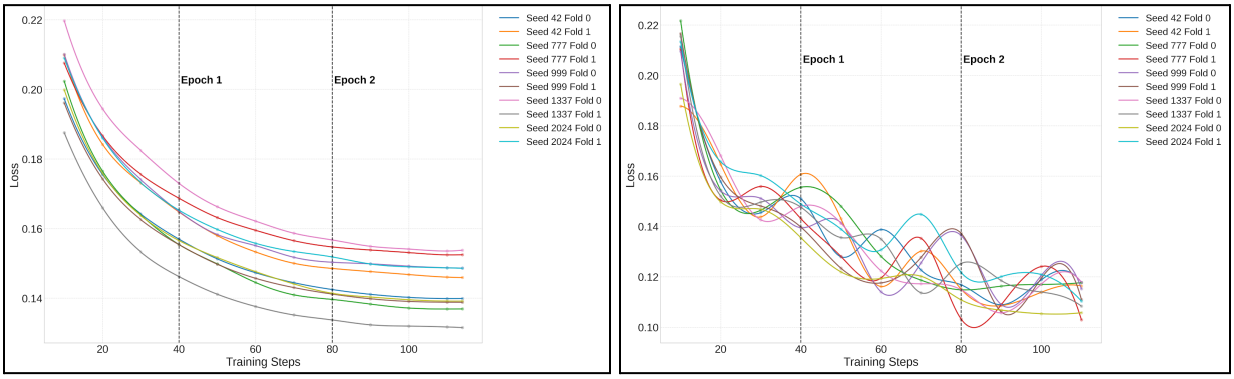

Figure 1: Proposed Model Training (left) and Evaluation (right) Loss Convergence

Model	ChRF	CodeBLEU	CodeBERT F1	Pass@1 Compile Rate
Baseline	57.27	51.29	88.04	42.53%
Proposed	65.89	58.41	90.39	62.17%

Figure 2: Evaluation Results by Metric Across 10 Folds

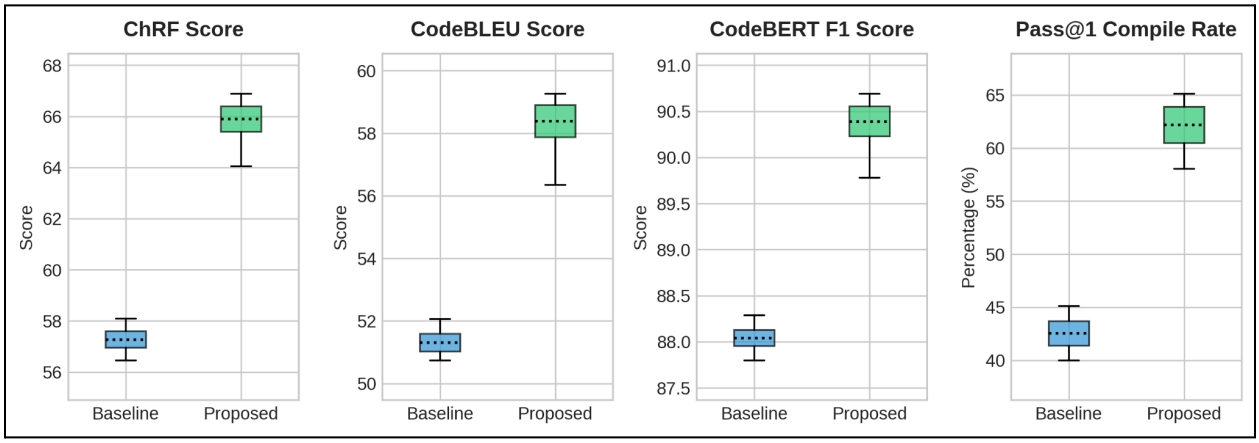

Figure 3: Evaluation Result Distributions by Metric Across 10 Folds

Metric	*t*-statistic	*p*-value	Significance (α = 0.01)
ChRF	8.859	0.0003	Significant
CodeBLEU	8.589	0.0004	Significant
CodeBERT F1	8.042	0.0005	Significant
Pass@1 Compile Rate	6.702	0.0011	Significant

Figure 4: Results of Dietterich 5x2cv Paired t-Test

As shown by the table in Figure 4, for all metrics tested, their *p*-values fall below the significance threshold, proving that $\mu_P > \mu_B$. Therefore, we reject the null hypothesis H_0 and accept the alternative hypothesis H_A. This proves that the improvements we see in our proposed model are statistically significant and genuine improvements over the baseline.

5. Discussion & Failure Modes

5.1 Performance Analysis

While analyzing our proposed model, we looked at the top 3 highest-ranked responses for each of the 10 proposed models we fine-tuned. From this, we were able to determine that our model is exceptionally good at translating stateless C functions that do not contain pointer manipulation into perfect idiomatic Rust. Our proposed model also has a good grasp of how to translate assembly code, file I/O, and string manipulation. However, our model shows minor variations in type and identifier name translation when compared to ground truth solutions.

Source C Code	Translated Rust Code

`#include <math.h>`
 	
`long long llrint(double x) {`
`  long long r;`
`  __asm__ ("fistpll %0" : "=m"(r) : "t"(x) : "st");`
`  return r;`
`}`
	`pub fn llrint(x: f64) -> i64 {`
	`  x.round_ties_even() as i64`
	`}`

Figure 5: Example of Assembly Found in C Translated to Idiomatic Rust

5.2 Error Analysis

While analyzing our proposed model, we also looked at the bottom 3 lowest-ranked responses for each of the 10 proposed models we fine-tuned. The insights these responses provided to us were far more useful than we expected. We discovered that the root cause of many of the poor translations traced back to one specific constraint we imposed upon our proposed model in its system instruction during fine-tuning and inference. The specific offending constraint was the one that stated: “Strictly avoid FFI types (`c_int`, `c_char`, `c_void`, `*mut T`).” While avoiding the use of raw pointer types is mandatory when attempting to convert C code to safe idiomatic Rust, avoiding the use of one-to-one primitive C type recreations is not. In fact, according to what we found, avoiding these types can actually be detrimental to safe translation.

The primary reason avoiding FFI types degrades the C to Rust translation of our proposed model is that it makes one-to-one functional parity impossible or impractical. From what we could see in the lowest-ranked translations in our evaluation results, many translations failed because the C code utilized types and functions, often from high-precision arithmetic libraries, that do not exist in standard Rust. Our model reacted to these non-existent types by re-inventing them from scratch. These re-inventions were often extremely over-engineered and, as a result, error-prone. Additionally, sometimes during long re-inventions, our model would collapse by entering a degenerate loop and outputting the same sequence of tokens over and over again.

Expected Rust Translation	Actual Rust Translation

`fn fac(self_: Func, n: Func) -> Func {`
`  let nn = n.num;`
`  if nn > 1 {`
`  let env = self_.env.as_ref().unwrap().upgrade().unwrap();`
`  num(nn * call(env, num(nn - 1)).num)`
`  } else {`
`  num(1)`
`  }`
`}`
	`fn fac(self_: &Func, _: &Func) -> Box<Func> {`
	`  let nn = self_.call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0)).call(num(0))`

Figure 6: Example of Degenerative Looping Caused by Re-Invention

6. Conclusion & Future Work

Overall, we nearly reached our goal of creating a C code to safe idiomatic Rust code transpiler. We initially aimed to achieve this goal by fine-tuning a CodeT5+ model, but due to extreme technical difficulty, we ultimately decided to switch to fine-tuning a Qwen2.5 Coder model instead. While in theory, the encoder-decoder architecture of the CodeT5+ model series should have been the most ideal for our use case, there were too many configuration issues with the CodeT5+ model series. After switching to Qwen2.5 Coder as our baseline model, we were able to fine-tune it for C to Rust code translation, but found its performance to be lacking. This lackluster performance was likely due to the small amount of data we were able to collect for training.

If we were to continue research and development of this project, there are three clear avenues for improvement that could be pursued. First, finding more training data would be the top priority. It is obvious that our dataset is too small to adequately adapt a reasonably sized LLM for C to Rust code translation; hence, more sources or training data would need to be identified. Second, giving the translation model access to an MCP server would likely increase the quality of translations. Examples of tools that could be provided to the model through the MCP server include a C compiler tool, a Rust compiler tool, and an abstract syntax tree analysis tool. Third, the project goal should shift toward semantic C to Rust code translation rather than line-by-line direct translation. This is because, while direct translation would be ideal, many fundamental differences between C and Rust make safe translation in all scenarios impossible. As a result, we believe that C to Rust translation should focus more on converting the concepts and ideas the C code is trying to convey rather than the C code itself.

References

[1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” openreview.net, Oct. 06, 2021. https://openreview.net/forum?id=nZeVKeeFYf9

[2] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” arXiv.org, May 20, 2023. https://arxiv.org/abs/2305.07922

[3] B. Hui et al., “Qwen2.5-Coder Technical Report,” arXiv.org, 2024. https://arxiv.org/abs/2409.12186